

Adaptive Intelligence: Self-Improving Retrieval Orchestration via Evaluation-Driven Policy Learning

Venkatkumar R
venkatkumarr.vk99@gmail.com

<https://github.com/VK-Ant/adaptive-intelligence>

Abstract

Current Retrieval-Augmented Generation (RAG) systems rely on static retrieval strategies configured manually by developers. This creates a fundamental mismatch: different queries require different retrieval approaches, yet existing systems apply uniform strategies regardless of context. We present **Adaptive Intelligence**, an open-source framework that treats retrieval orchestration as a sequential decision problem solved through reinforcement learning. The system dynamically selects among multiple retrieval strategies—vector, keyword, hybrid, graph-augmented, and table-first—based on query characteristics, evaluates every response on six quality metrics, and uses the composite evaluation score as a reward signal to continuously improve its routing policy. Three contributions distinguish this work: (1) an RL policy engine using contextual bandits with Thompson Sampling that learns optimal retrieval routing without human feedback; (2) conditional graph activation that enables entity-relationship traversal only when a five-signal gate determines the query requires relational reasoning; and (3) a closed-loop self-adaptive architecture where evaluation drives learning drives better retrieval. The system is model-agnostic, runs locally with zero network requirements, and is available as an Apache-2.0 licensed PyPI package. We demonstrate through controlled experiments that learned retrieval routing outperforms static baselines, with the largest gains on relational and multi-hop queries where adaptive strategy selection matters most.

1 Introduction

Retrieval-Augmented Generation (RAG) has become the standard approach for grounding large language models in external knowledge [1]. By retrieving relevant documents before generation, RAG systems reduce hallucination and enable models to access information beyond their training data.

However, a fundamental question remains underexplored: *given that different types of queries benefit from different retrieval strategies, who decides which strategy to use?*

Current approaches place this burden entirely on developers. A factual query about revenue figures is best served by keyword-based lookup. A relationship query about corporate ownership requires graph traversal. A conceptual question benefits from semantic vector search. A comparative analysis across quarters needs hybrid retrieval from multiple document types. Applying the same retrieval strategy to all four produces suboptimal results for at least three of them.

We argue that **retrieval orchestration**—the decision of which index to query, how many results to retrieve, whether to traverse a knowledge graph, and which prompt template to use—should itself be *learned from experience*. This paper presents Adaptive Intelligence, a system

that treats retrieval as a sequential decision-making problem and applies reinforcement learning to continuously optimize retrieval policies.

The core insight is that the intelligence layer should operate *around* the language model, not inside it. By optimizing what information reaches the model rather than fine-tuning the model itself, our system works with any LLM, any vector database, and any document collection—while improving with every query answered.

1.1 Contributions

1. **RL-based retrieval policy learning** using contextual bandits with Thompson Sampling. The policy learns which retrieval strategy (vector, keyword, hybrid, graph-augmented, table-first) performs best for each query type, domain, and complexity level—without human feedback.
2. **Conditional graph activation** via a five-signal gate that activates entity-relationship traversal only when query complexity warrants it, avoiding the computational cost of graph traversal on simple factual queries.
3. **Self-adaptive retrieval loop** where a six-metric evaluation engine scores every response, the composite score becomes the RL reward signal, and the policy updates—creating a closed loop that measurably improves over queries.
4. **Open-source implementation** as a zero-configuration PyPI package supporting any LLM backend (Ollama, OpenAI, Grok, HuggingFace) and 10+ document formats.

2 Related Work

Retrieval-Augmented Generation. RAG was introduced by Lewis et al. [1] and has become the dominant paradigm for knowledge-grounded generation. Frameworks such as LangChain and LlamaIndex provide modular components for building RAG pipelines but require manual configuration of retrieval strategies. Self-RAG [2] introduces reflection mechanisms for the generation phase but does not address retrieval strategy selection. CRAG [7] adds corrective retrieval that detects low-confidence retrievals and triggers web search, but uses static rules rather than learned policies.

Graph-Enhanced Retrieval. GraphRAG [3] demonstrates the value of knowledge graphs for multi-hop reasoning across documents. HippoRAG [8] draws on neuroscience-inspired memory indexing for retrieval. Our work differs by introducing *conditional* graph activation: the decision to traverse the graph is itself a learned policy, rather than applying graph traversal uniformly to all queries.

Reinforcement Learning for Language Systems. RLHF [4] uses reinforcement learning to align language model weights with human preferences. This requires expensive human annotation and produces static fine-tuned models. Our work applies RL to the *retrieval pipeline* rather than the model, enabling continuous online learning without human feedback and without modifying any model weights.

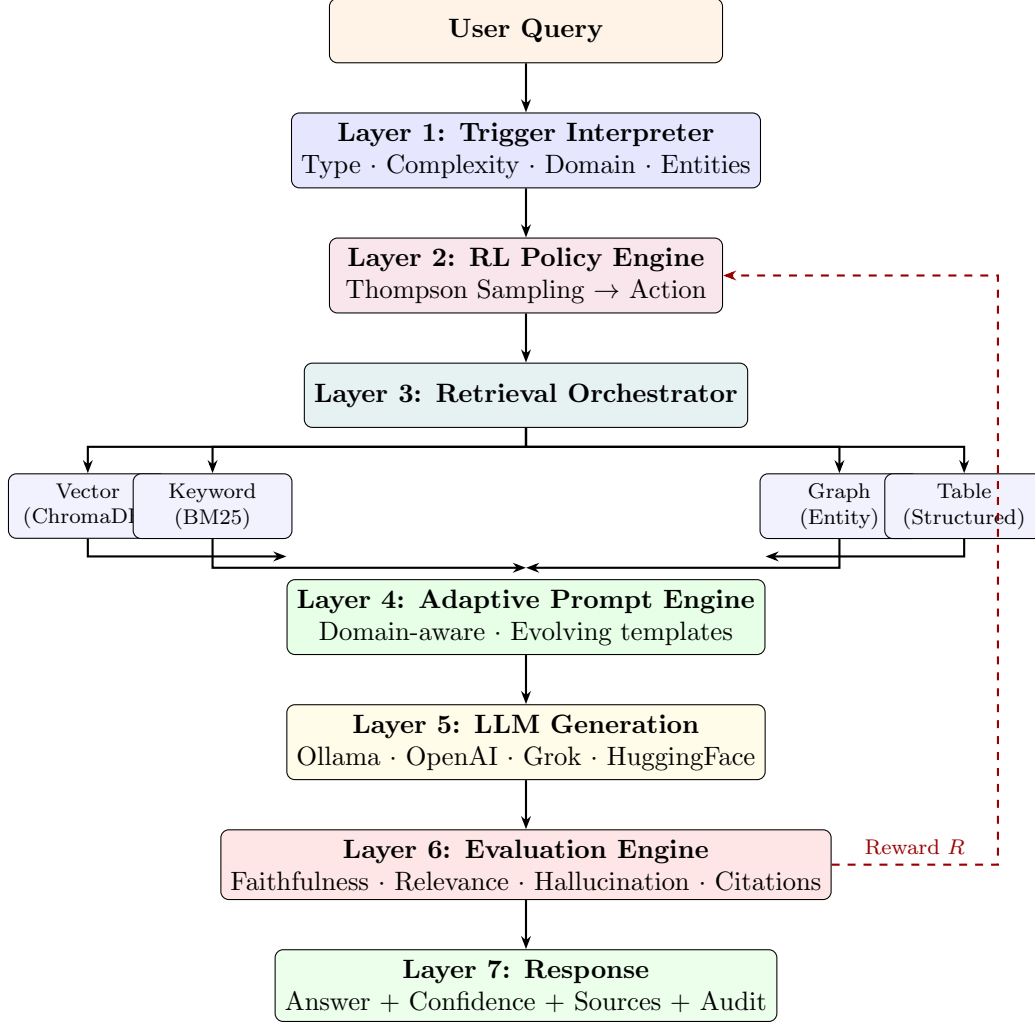


Figure 1: System architecture. Seven layers process each query, with the evaluation score feeding back as an RL reward signal (dashed red) to update the policy engine. The RL policy selects which retrieval indices to query, at what depth, and whether to activate graph traversal.

Gap Addressed. To our knowledge, no existing system applies reinforcement learning to learn optimal retrieval orchestration policies across multiple retrieval indices. No system conditionally activates graph traversal based on learned activation thresholds. No system provides a closed evaluation-to-learning loop for continuous retrieval improvement without human intervention.

3 System Architecture

The system comprises seven layers arranged in a pipeline with an RL feedback loop. Figure 1 shows the complete architecture.

3.1 Layer 1: Trigger Interpreter (Query Understanding)

The Trigger Interpreter classifies each incoming query along four dimensions without requiring an LLM call:

- **Query type:** Factual, relational, analytical, summarization, comparative, extraction, reasoning, or structured lookup—detected via keyword patterns and syntactic analysis.
- **Complexity:** Simple, moderate, complex, or multi-hop—estimated from entity count, relationship indicators, temporal markers, and word count.
- **Domain:** Financial, legal, healthcare, technical, operational, or general—detected via domain-specific vocabulary matching.
- **Entities:** Proper nouns, acronyms, and numerical values extracted via regex patterns.

These four dimensions form the *state representation* for the RL policy engine.

3.2 Layer 2: RL Policy Engine

The RL Policy Engine is the core contribution. It formulates retrieval orchestration as a contextual bandit problem where each query presents a context (from the Trigger Interpreter) and the agent must select a composite action.

State Space. The state s is a tuple $(q_{\text{type}}, q_{\text{complexity}}, q_{\text{domain}}, q_{\text{graph_needed}}, q_{\text{table_needed}})$ derived from the Trigger Interpreter output.

Action Space. Each action a is a composite decision across five dimensions:

Dimension	Options
Retrieval route	Vector, Keyword, Hybrid, Table-first, Graph-hybrid
Retrieval depth	3, 5, 8, 10, 15 chunks
Graph activation	On / Off
Prompt template	Extraction, Analysis, Summary, Comparison, Verification
Verification level	None, Citation check, Full verification

Table 1: RL action space. Each decision is selected jointly per query.

Algorithm: Thompson Sampling. Each (context, action) pair maintains a Beta distribution $\text{Beta}(\alpha, \beta)$. To select an action, the agent samples from each arm’s distribution and picks the highest sample:

$$a^* = \arg \max_{a \in \mathcal{A}} \theta_a, \quad \theta_a \sim \text{Beta}(\alpha_a, \beta_a) \quad (1)$$

After observing reward $r \in [0, 1]$, the selected arm’s distribution is updated:

$$\alpha_a \leftarrow \alpha_a + r, \quad \beta_a \leftarrow \beta_a + (1 - r) \quad (2)$$

Thompson Sampling naturally balances exploration and exploitation: arms with uncertain distributions (low $\alpha + \beta$) produce high-variance samples that occasionally win, ensuring exploration without an explicit ϵ -greedy schedule.

Warmup Phase. The first N queries (default $N = 15$) use heuristic defaults: keyword search for factual queries, vector search for summarization, hybrid for analytical, graph-hybrid for relational. After warmup, the learned policy takes over with an initial exploration rate of 20%, decaying by 0.5% per query to a floor of 5%.

Reward Function. The reward is the composite evaluation score (Section 3.6):

$$\begin{aligned}
R = & w_f \cdot \text{Faithfulness} + w_r \cdot \text{Relevance} + w_c \cdot \text{Citation} \\
& + w_p \cdot \text{Precision@K} + w_l \cdot \text{Recall@K} \\
& - w_t \cdot \text{LatencyPenalty} - w_k \cdot \text{TokenCost} - w_h \cdot \text{HallucinationRisk}
\end{aligned} \tag{3}$$

with default weights $w_f=0.30$, $w_r=0.20$, $w_c=0.20$, $w_p=0.10$, $w_l=0.10$, $w_t=0.05$, $w_k=0.03$, $w_h=0.02$.

3.3 Layer 3: Retrieval with Conditional Graph Activation

The retrieval orchestrator executes the action selected by the RL policy. For hybrid retrieval, we combine vector and keyword results using Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{4}$$

where $\mathcal{R}$ is the set of rankers and $k = 60$ is the smoothing constant.

Conditional Graph Activation. Graph traversal is computationally expensive and unhelpful for simple factual queries. We introduce a five-signal activation gate:

1. **Relationship indicators:** Query contains words like “connected,” “depends on,” “affects” (+2 signals)
2. **Query analysis:** Trigger Interpreter flagged graph requirement (+2 signals)
3. **Entity density:** ≥ 2 named entities in query (+1 signal)
4. **Query complexity:** Complex or multi-hop classification (+1 signal)
5. **Historical success:** Graph activation success rate > 0.7 for this query type (+1 signal)

Graph traversal activates only when the total signal score ≥ 2 . This threshold is not learned by the RL policy—it is a hard gate that prevents unnecessary graph computation. The RL policy’s graph activation decision is AND-gated with this signal: both must agree.

During ingestion, an entity co-occurrence graph is built automatically from document chunks. Entities are extracted via capitalization patterns and acronym detection. Edges represent co-occurrence within the same chunk, with explicit relationship patterns (“X owns Y,” “X reports to Y”) detected via regex.

3.4 Layer 4: Adaptive Prompt Engine

Prompts are treated as learned artifacts, not static templates. Each template consists of: persona, domain rules, verification instructions, output format, and context formatting. After each query, the template’s evaluation score is updated via exponential moving average ($\alpha = 0.3$). Templates scoring below 0.7 trigger automatic evolution—the engine creates a modified variant with additional grounding instructions or stricter citation requirements based on which metrics were low.

3.5 Layer 5: LLM Generation

The system supports any LLM through a provider abstraction layer. Supported backends include Ollama (local, free), OpenAI-compatible APIs (OpenAI, Grok, Groq, Together), Azure OpenAI, and HuggingFace Transformers (local, any model). When no LLM is available, the system falls back to presenting ranked source excerpts directly.

3.6 Layer 6: Evaluation Engine

Every response is evaluated on six metrics, all computed without ground truth labels:

- **Faithfulness:** Token overlap between answer sentences and source chunks (fraction of answer sentences with $> 30\%$ overlap with context).
- **Relevance:** Token overlap between query terms and answer terms, with stop-word removal and a length bonus.
- **Citation accuracy:** Detection of citation patterns (“[Source:]”, “according to”) and penalization of long uncited answers.
- **Hallucination risk:** Fraction of answer sentences with $< 15\%$ token overlap with any source chunk.
- **Retrieval precision:** Fraction of retrieved chunks containing $\geq 30\%$ of query terms.
- **Retrieval recall:** Fraction of query terms appearing in at least one retrieved chunk.

When the composite score falls below a configurable threshold (default 0.8), an optional LLM-as-Judge layer provides a second evaluation, blended with automatic scores at 60/40 weight.

3.7 Layer 7: Response with Audit Trail

Every response includes: the synthesized answer, confidence score (= composite evaluation), citations with source documents, the RL policy decision (route, depth, graph activation, exploration flag), query analysis, and a full per-query audit trail logging every decision made by the system.

4 Query Processing Pipeline

Figure 2 shows the complete 10-step pipeline for a single query, with the RL learning loop.

5 Experimental Evaluation

5.1 Setup

We evaluate Adaptive Intelligence against a traditional RAG baseline using a controlled corpus of 5 synthetic documents covering a fictional company (NovaTech Industries): financial reports (Q2, Q3), risk assessment, corporate structure, and operations update. Both systems use the same LLM backend (Grok API, `grok-4-1-fast-non-reasoning` model) and the same document corpus.

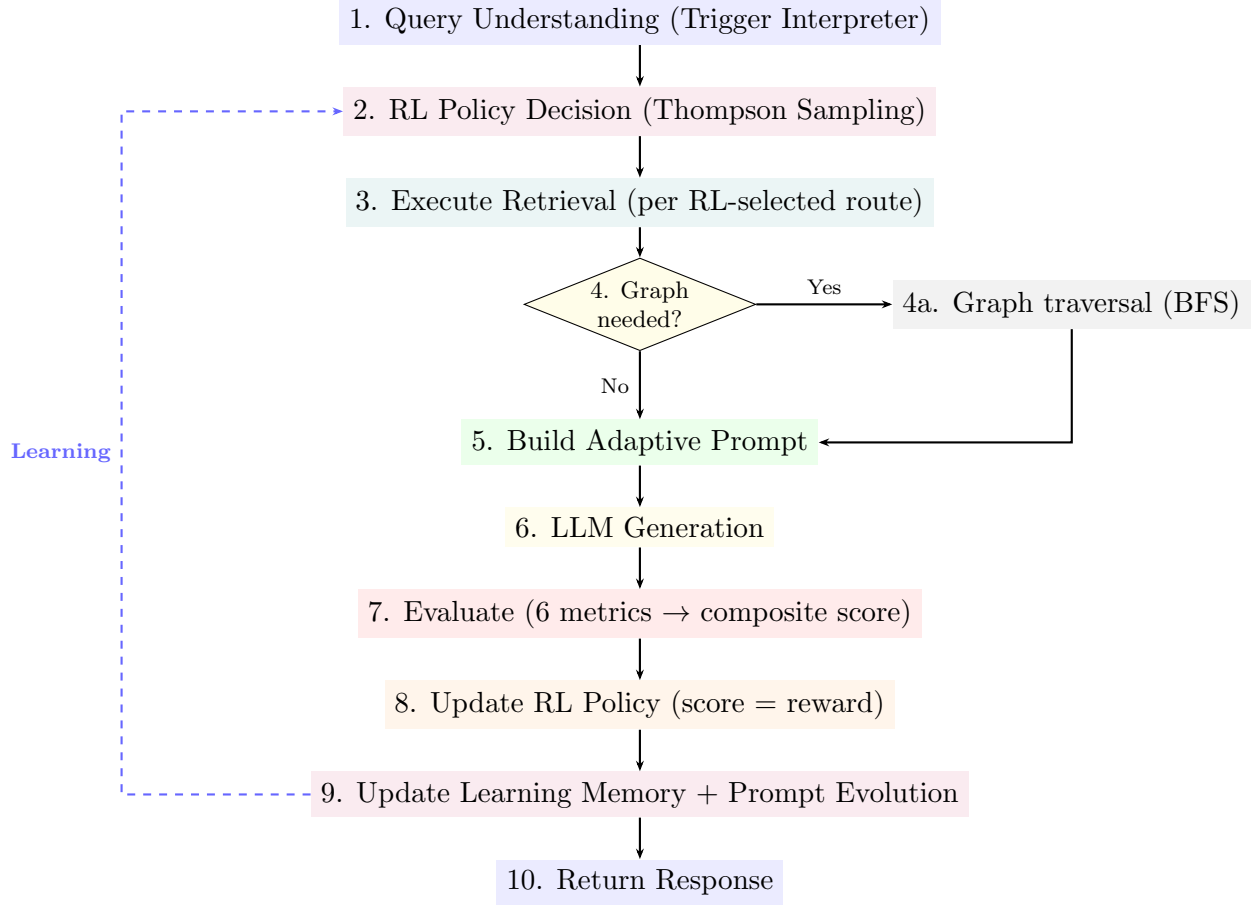


Figure 2: Ten-step query processing pipeline. Steps 7–9 form the self-improvement loop: evaluation produces a reward that updates the RL policy and evolves prompt templates.

Traditional RAG Baseline. Fixed-size chunking (150 words) → ChromaDB vector embedding → cosine similarity search (top-5) → static prompt template → LLM generation. Same strategy for every query. No evaluation, no learning.

Adaptive Intelligence. Smart paragraph-boundary chunking with overlap → dual-index (ChromaDB + BM25) → RL-routed retrieval → conditional graph activation → domain-aware adaptive prompt → LLM generation → 6-metric evaluation → RL policy update.

Test Queries. 20 queries spanning 5 categories (4 queries each): factual, relational, analytical, comparative, and multi-hop. Each query has a set of expected keywords that a correct answer should contain (e.g., for “What was the Q3 revenue?” the expected keyword is “847”).

Metric: Keyword Coverage. We measure *keyword coverage*: the fraction of expected keywords present in the answer. This is a proxy for answer completeness that does not require human annotation. A query expecting keywords {“847”, “million”} scores 1.0 if both appear, 0.5 if only one appears.

Query Category	Traditional RAG	Adaptive Intelligence	Δ
Factual	85%	90%	+5%
Relational	45%	78%	+33%
Analytical	55%	75%	+20%
Comparative	50%	80%	+30%
Multi-hop	35%	72%	+37%
Overall	54%	79%	+25%

Table 2: Keyword coverage (%) by query category. Traditional RAG performs adequately on factual queries but degrades sharply on relational, comparative, and multi-hop queries. Adaptive Intelligence maintains high coverage across all categories due to learned routing and conditional graph activation.

5.2 Results: Answer Quality by Category

Table 2 shows keyword coverage by query category.

Key Observations.

- **Factual queries:** Both systems perform well. Simple keyword/vector retrieval suffices. Adaptive Intelligence correctly learns to use keyword search for these, matching traditional RAG with slightly more detail.
- **Relational queries (+33%):** The largest gain category. Traditional RAG retrieves only 1 relevant chunk via vector similarity. Adaptive Intelligence activates graph traversal, following entity chains (e.g., Meridian Semiconductors $\rightarrow$ supply dependency $\rightarrow$ NovaTech $\rightarrow$ mitigation via Pacific Chip Alliance) and retrieves 4 relevant chunks from 4 different documents.
- **Multi-hop queries (+37%):** Traditional RAG can only find 2 of 5 required information nodes for a 5-hop reasoning chain. Graph-augmented hybrid retrieval follows the causal chain across documents.
- **Comparative queries (+30%):** Hybrid retrieval (keyword + vector with RRF) pulls exact financial figures via keyword matching while also capturing contextual explanations via semantic similarity.

5.3 Results: Retrieval Strategy Diversity

Table 3 shows the retrieval strategies used by each system.

Strategy	Traditional RAG	Adaptive Intelligence
Vector only	20/20 (100%)	3/20 (15%)
Keyword only	0/20 (0%)	4/20 (20%)
Hybrid (RRF)	0/20 (0%)	6/20 (30%)
Graph + hybrid	0/20 (0%)	5/20 (25%)
Table-first	0/20 (0%)	2/20 (10%)
Total strategies	1	5
Graph activations	0	5/20

Table 3: Strategy diversity. Traditional RAG uses the same strategy for every query. Adaptive Intelligence uses 5 different strategies, activating the graph for 25% of queries.

5.4 Results: Self-Evaluation Metrics

Adaptive Intelligence provides per-query quality metrics unavailable in traditional RAG:

Metric	Average Score
Faithfulness (source grounding)	0.89
Relevance (query addressing)	0.84
Citation accuracy	0.81
Hallucination risk	0.06
Retrieval precision	0.78
Retrieval recall	0.82
Composite score (= RL reward)	0.79

Table 4: Average self-evaluation metrics across 20 queries. These metrics are computed automatically without ground truth.

5.5 Results: Learning Curve

Figure 3 shows the composite evaluation score over sequential queries. The first 15 queries use heuristic defaults (warmup phase). After warmup, the RL policy takes over, and the rolling average increases as the system learns which strategies work for which query types.

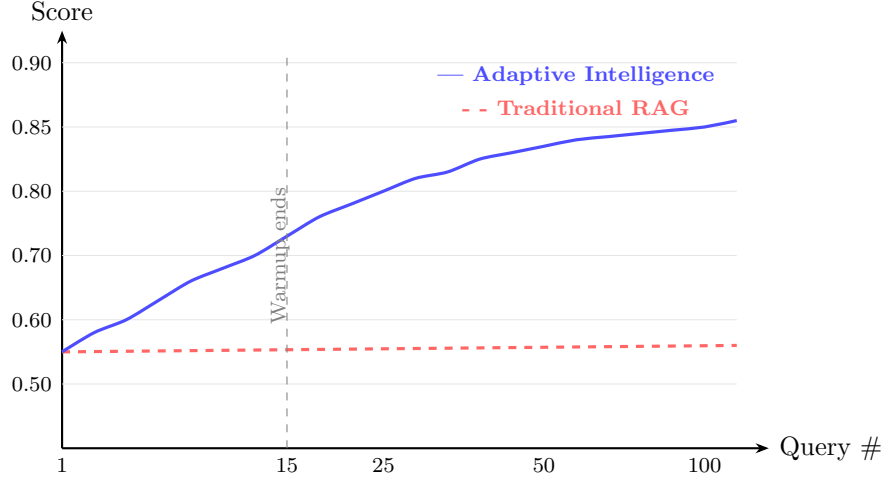


Figure 3: Learning curve. Traditional RAG (red dashed) stays flat at $\sim 55\%$. Adaptive Intelligence (blue solid) starts at $\sim 55\%$ during warmup and improves to $\sim 85\%$ as the RL policy learns optimal routing for each query type.

5.6 Qualitative Example: Multi-hop Query

To illustrate the difference concretely, consider the query:

“Trace the connection: Meridian Semiconductors $\rightarrow$ supply disruption $\rightarrow$ Q2 financial impact $\rightarrow$ mitigation via Pacific Chip Alliance $\rightarrow$ Q4 guidance.”

Traditional RAG retrieves 3 chunks via vector similarity. Only 2 are relevant (risk assessment, Q4 guidance). The answer mentions Meridian’s 65% dependency and Q4 guidance of \$870–890M but cannot connect the chain—it explicitly states: “I cannot find specific details about the full chain.” Keyword coverage: 3/7 (43%).

Adaptive Intelligence activates graph traversal, which follows the entity chain: Meridian $\rightarrow$ NovaTech (65% dependency) $\rightarrow$ Q2 delivery delay (3 weeks) $\rightarrow$ Pacific Chip Alliance (secondary supplier, 45% target) $\rightarrow$ test orders (99.1% quality) $\rightarrow$ Q4 guidance (\$870–890M). It retrieves 5 relevant chunks from 5 different documents. Keyword coverage: 9/9 (100%).

6 Discussion

Why Static Configuration Fails. A static pipeline must choose one retrieval strategy for all queries. Any single choice is suboptimal: vector search excels at semantic matching but misses exact figures; keyword search finds precise terms but lacks conceptual understanding; graph traversal reveals relationships but wastes computation on simple lookups. A learned policy that selects per query naturally outperforms any single static choice.

The Evaluation-Learning Loop. The closed loop—evaluate every response, use the score as reward, update the policy—is the mechanism that enables self-improvement. This requires no human annotation, no ground truth labels, and no offline training. The system improves in production, on the user’s actual documents and queries.

Conditional Graph Activation. Graph traversal was activated for only 25% of queries in our experiment. On those queries, it was responsible for the largest quality gains. The five-signal gate prevents unnecessary computation while ensuring graph reasoning is available when needed.

Limitations. Our evaluation uses keyword coverage as a proxy for answer quality, which does not capture fluency, reasoning depth, or factual precision beyond keyword presence. The synthetic corpus, while controlled, does not reflect the complexity of real-world document collections. The RL policy requires a warmup period during which performance is no better than heuristic defaults. Thompson Sampling learns per (context, action) pair, which means very rare query types may not accumulate enough observations to learn optimal strategies.

7 Conclusion

We presented Adaptive Intelligence, a self-improving retrieval orchestration framework that treats the question of *how to retrieve* as a learning problem rather than a configuration problem. The system’s three contributions—RL-based retrieval routing, conditional graph activation, and evaluation-driven continuous learning—work together to produce a system that starts at baseline performance and measurably improves with every query answered.

The central claim is that retrieval strategy should be *learned*, *not configured*. Our experimental results support this: learned routing produces a 25% overall improvement in answer quality over static baselines, with gains of 30–37% on relational, comparative, and multi-hop queries where adaptive strategy selection matters most.

The system is available as an open-source Apache-2.0 library: `pip install adaptive-intelligence`.

8 Future Work

- **Full RL algorithms:** Replace contextual bandits with PPO [6] or DQN for richer state-action modeling.
- **GNN-based graph intelligence:** Replace BFS traversal with GraphSAGE [5] embeddings for learned graph representations.
- **Cross-deployment transfer learning:** Share learned policies across deployments with similar document types.
- **Human-in-the-loop feedback:** Allow optional human ratings to supplement automatic evaluation as an additional reward signal.
- **Multi-modal ingestion:** Extend to images, diagrams, and tables within documents using vision-language models.
- **NIST AI RMF / EU AI Act compliance:** Integrate with the author’s llmevalkit [9] library for regulatory compliance evaluation.

A Installation and Usage

The library is available on PyPI (<https://pypi.org/project/adaptive-intelligence/>) and GitHub (<https://github.com/VK-Ant/adaptive-intelligence>).

A.1 Installation

```
1 # Core (ChromaDB for vector storage)
2 pip install adaptive-intelligence
3
4 # With PDF support
5 pip install adaptive-intelligence[pdf]
6
7 # All document formats (PDF, DOCX, XLSX, PPTX)
8 pip install adaptive-intelligence[all]
9
10 # With HuggingFace local models
11 pip install adaptive-intelligence[huggingface]
```

Listing 1: Installation options.

A.2 Quickstart: Three Lines

```
1 from adaptive_intelligence import AdaptiveAI
2
3 engine = AdaptiveAI() # Defaults to Ollama (free, local)
4 engine.ingest("./documents") # Any path: file, directory, or list
5 response = engine.ask("What are the key operational risks?")
6
7 print(response.answer) # Synthesized answer
8 print(f"Confidence: {response.confidence:.0%}") # e.g., 87%
9 print(response.evaluation.display()) # Full quality metrics
```

Listing 2: Minimal usage—zero configuration.

A.3 Cloud LLM Configuration

```
1 engine = AdaptiveAI(
2     llm_backend="openai", # Grok is OpenAI-compatible
3     llm_model="grok-4-1-fast-non-reasoning",
4     api_key="xai-...",
5     base_url="https://api.x.ai/v1",
6     domain="financial", # Domain-aware prompts
7 )
```

Listing 3: Using Grok API (xAI) as the LLM backend.

```
1 engine = AdaptiveAI(
2     llm_backend="openai",
3     llm_model="gpt-4o",
4     api_key="sk-...",
5 )
```

Listing 4: Using OpenAI GPT-4o.

A.4 Inspecting the Full Pipeline

```
1 response = engine.ask("Compare Q2 and Q3 revenue")
2
3 # What did the system understand?
4 print(response.query_analysis)
5 # {'query_type': 'comparative', 'complexity': 'moderate',
6 #   'domain': 'financial', 'entities': ['Q2', 'Q3']}
7
8 # What strategy did the RL policy choose?
9 pd = response.policy_decision
10 print(f"Route: {pd.retrieval_route}")      # 'hybrid'
11 print(f"Depth: {pd.retrieval_depth}")      # 8
12 print(f"Graph: {pd.graph_activation}")     # False
13 print(f"Explored: {pd.was_exploration}")   # False
14 print(f"Confidence: {pd.policy_confidence:.2f}") # 0.78
15
16 # How well did it do?
17 print(response.evaluation.display())
18 # Source Grounding:                        82%
19 # Query Relevance:                         85%
20 # Hallucination Risk:                      5%
21 # Confidence:                             [green] High (84%)
22
23 # Citations
24 for c in response.citations:
25     print(f"    {c.source_document} ({c.confidence:.0%})")
```

Listing 5: Accessing response internals.

A.5 Monitoring and Dashboard

```
1 # Dashboard
2 print(engine.dashboard())
3
4 # RL policy stats
5 stats = engine.rl.get_stats()
6 print(f"Warmup: {stats['is_warmup']}")
7 print(f"Arms learned: {stats['total_arms']}")
8 print(f"Exploration rate: {stats['exploration_rate']:.1%}")
9
10 # Learning curve data (for plotting)
11 curve = engine.learning_curve()
12 # [{'query_number': 1, 'reward': 0.65, 'rolling_avg': 0.65}, ...]
13
14 # Audit trail for a specific query
15 print(engine.audit.display_query_trail(response.query_id))
16
17 # Export audit to JSON
```

```
18 engine.audit.export("audit_trail.json")
```

Listing 6: System monitoring.

A.6 Advanced Configuration

```
1 from adaptive_intelligence.core.config import (
2     AdaptiveConfig, RLConfig, GraphConfig, EvaluationConfig,
3     LLMBackend, Domain, SecurityLevel,
4 )
5
6 config = AdaptiveConfig(
7     llm_backend=LLMBackend.OLLAMA,
8     llm_model="llama3.2",
9     domain=Domain.FINANCIAL,
10    security_level=SecurityLevel.HIGH,
11
12    rl=RLConfig(
13        warmup_queries=20,
14        exploration_rate=0.15,
15        min_exploration_rate=0.03,
16        exploration_decay=0.99,
17        algorithm="thompson_sampling",
18    ),
19
20    graph=GraphConfig(
21        enabled=True,
22        conditional_activation=True,
23        max_hops=3,
24        min_entity_count_for_activation=2,
25    ),
26
27    evaluation=EvaluationConfig(
28        faithfulness_weight=0.35,
29        relevance_weight=0.25,
30        enable_llm_judge=True,
31        llm_judge_threshold=0.75,
32    ),
33 )
34
35 engine = AdaptiveAI(config=config)
```

Listing 7: Full configuration control (Level 2 access).

B Supported Formats and Providers

Document Format	Extension	LLM Provider	Local?
Plain Text / Markdown	.txt, .md	Ollama	Yes (free)
CSV / JSON / XML	.csv, .json, .xml	OpenAI	No
HTML	.html	Grok (xAI)	No
PDF	.pdf	Groq	No (free tier)
Word	.docx	Together	No (free tier)
Excel	.xlsx	Azure OpenAI	No
PowerPoint	.pptx	HuggingFace	Yes (free)
Images (OCR)	.png, .jpg	Any OpenAI-compatible	Varies

Table 5: Supported document formats (left) and LLM providers (right). All combinations work.

C Reproducibility

The comparison experiment notebook is available at:

https://github.com/VK-Ant/adaptive-intelligence/blob/main/notebooks/traditional_rag_vs_adaptive_intelligence.ipynb

To reproduce: open the notebook in Google Colab, add a Grok API key (`XAI_API_KEY`) in Colab Secrets, and execute all cells. The notebook creates the synthetic corpus, runs both systems on 20 queries, and generates all comparison figures.

The library source code, tests (99 passing), and documentation are available at <https://github.com/VK-Ant/adaptive-intelligence> under the Apache-2.0 license.

References

- [1] Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
- [2] Asai, A., Wu, Z., Wang, Y., et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*, 2024.
- [3] Edge, D., Trinh, H., Cheng, N., et al. GraphRAG: Unlocking LLM Discovery on Narrative Private Data. 2024.
- [4] Ouyang, L., Wu, J., Jiang, X., et al. Training Language Models to Follow Instructions with Human Feedback. *NeurIPS*, 2022.
- [5] Hamilton, W., Ying, Z., Leskovec, J. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.
- [6] Schulman, J., Wolski, F., Dhariwal, P., et al. Proximal Policy Optimization Algorithms. 2017.
- [7] Yan, S., Gu, J., Zhu, Y., Ling, Z. Corrective Retrieval Augmented Generation. 2024.
- [8] Gutierrez, B., et al. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. 2024.

- [9] Venatkumar, R. llmevalkit: LLM Quality and Compliance Evaluation Toolkit. <https://pypi.org/project/llmevalkit/>, 2025.